system, except that it returns the process ID of the spawned process and does not wait for the process to be finished to move forward.

One problem with system is that the command's output will simply go to the same destination as the program's output, which may not be what you want. To capture the standard output of a subprocess, you can use the backquote characters, as with date in the previous example. Note that you may need to use chomp to remove the line-ending characters from the result.

This is fine for simple cases—we can run an external process and get the return status. But many times we need a bit more control than that. We'd like to carry on a conversation with the subprocess, possibly sending it data and possibly getting some back. popen does just this. The popen method runs a command as a subprocess and connects that subprocess's standard input and standard output to a Ruby IO object. Write to the IO object, and the subprocess can read it on standard input. Whatever the subprocess writes is available in the Ruby program by reading from the IO object.

For example, on our systems one of the more useful utilities is pig, a program that reads words from standard input and prints them in pig latin (or igpay atinlay). We can use this when our Ruby programs need to send us output that our five-year-olds shouldn't be able to understand:

```
pig = IO.popen("local/util/pig", "w+")
pig.puts "ice cream after they go to bed"
pig.close_write
puts pig.gets
```

*produces:*

```
iceway eamcray afterway eythay ogay otay edbay
```

This example illustrates both the apparent simplicity and the more subtle real-world complexities involved in driving subprocesses through pipes. The code certainly looks simple enough: open the pipe, write a phrase, and read back the response. But it turns out that the pig program doesn't flush the output it writes. Our original attempt at this example, which had a pig.puts followed immediately by a pig.gets, hung forever. The pig program processed our input, but its response was never written to the pipe. We had to insert the pig.close_write line. This sends an end-of-file to pig's standard input, and the output we're looking for gets flushed as pig terminates.

The popen method has one more twist. If the command you pass it is a single minus sign (-), popen will fork a new Ruby interpreter. Both this and the original interpreter will continue running by returning from the popen. The original process will receive an IO object back, and the child will receive nil. This works only on operating systems that support the fork call[1] (and for now this excludes Windows, unless you use WDSL).

```
new_pipe = IO.popen("-","w+")
if new_pipe
  new_pipe.puts "Get a job!"
  $stderr.puts "I'm the parent, the child said to me '#{new_pipe.gets.chomp}'"
else
  $stderr.puts "I'm the child, the parent said to me '#{gets.chomp}'"
  puts "OK"
end
```

1. https://www.freebsd.org/cgi/man.cgi?query=fork